

DECENTRACAM

Abstract. A camera app that hashes the image and immutably records the hash onto the blockchain immediately after image capture. Image manipulation is increasingly hard to detect due to AI. This app prevents such manipulated images from being falsely verified as authentic by certifying the image at capture time. The phone's TEE signs the image hash using a private key attested to the authorized app, and hence ensures that the user cannot submit non-authentic or AI-generated images. The blockchain then validates the signature and allows the hash to be stored on-chain. This also reduces on-chain verification costs.

INTRODUCTION

The authenticity of images and videos is becoming increasingly precarious with the advent of AI generated images and advanced editing. This creates an urgent need for everyday users to easily verify critical images, and to capture images that can be published and universally accepted as authentic.

Current centralized solutions, such as Truepic Vision, suffer from several fundamental flaws. The most obvious is that they store the supposedly authentic image on centralized servers, creating a single point of failure. They alone verify image authenticity, making them a centralized and trusted third party. There are also serious privacy concerns, as the image and metadata (including GPS location) are handed over to the company, something many users find unacceptable, as we can see from the reviews on their app.

In courts today, forensic experts must painstakingly analyze images to establish authenticity, a process both time-consuming and uncertain.

What we need is a privacy-preserving, fully decentralized method to verify images at the moment of capture and upload cryptographic proof to the blockchain, making it immutable. Verification must move from trusted third parties to a trustless, on-chain environment. This can be achieved by uploading image hashes to the blockchain at capture time, with the TEE providing attestation of the image's source.

Journalists and publishers must attach the original lossless image (e.g., PNG or TIFF) alongside the proof for it to hold evidentiary value, since otherwise the hash will change causing verification to fail.

DecentraCam is optimized for Solana Seeker devices, which provide the hardware-backed trust guarantees necessary for real-time proof-of-origin generation. With StrongBox-enabled TEE keys, Verified Boot support, modern Camera2 APIs and secure camera HAL,

Seeker is uniquely positioned to run the DecentraCam pipeline securely and efficiently. Other Android phones may lack required attestation support or suffer performance issues during zero-knowledge proof generation.

System Architecture

The system is divided into three components that collaborate to ensure image authenticity.

1. **Application layer**
2. **Trusted execution environment (TEE)**
3. **Blockchain layer**

Application layer

This is the layer the user directly interacts with. The process from image capture to blockchain submission is fully automated and outside the user's control, ensuring integrity.

The application layer is where:

- A. On first launch, the app creates a private key inside the device's TEE via Android Keystore. The corresponding attestation certificate chain is generated and securely stored within the app.
- B. A zero-knowledge proof is generated locally (within the app) verifying the validity of an attestation, verifying that the attestation links the key to the authorized app package, and linked public key. This public key along with the zkproof is sent to the blockchain for verification. (public key gets stored on chain if verification passes) This step binds the public key to the user's wallet address in the smart contract, establishing the only valid signing key for future submissions.
- C. The image is captured along with metadata by the user
- D. Image hash is generated and the app requests the TEE to sign the image hash using the previously attested private key.
- E. A transaction is constructed containing a signature over (image hash || monotonic counter || user wallet public key), made using the previously attested private key. Which is then sent to the blockchain. If the signature is valid for the stored public key, the hash is immutably stored.

Trusted execution environment (TEE)

At initialization, the app uses the Android Keystore via the TEE to generate an app-scoped private key. Now using this private key we generate an attestation as to the source of the hash.

The TEE layer is where:

- A. Generating the attestation, which cryptographically links a private key to the authorized app
- B. The image hash, user wallet public key, and monotonic counter are signed by the attested private key and returned to the app

Blockchain layer

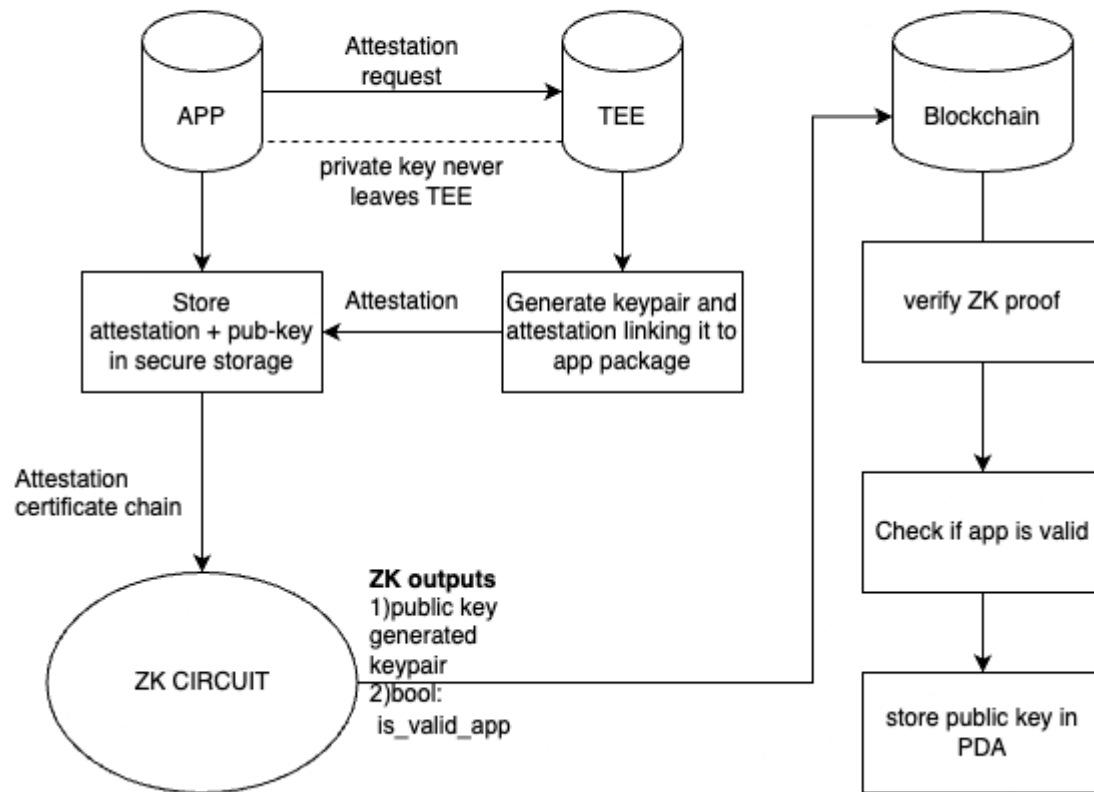
This layer is where the hash will finally be immutably stored once verification of its authenticity is complete.

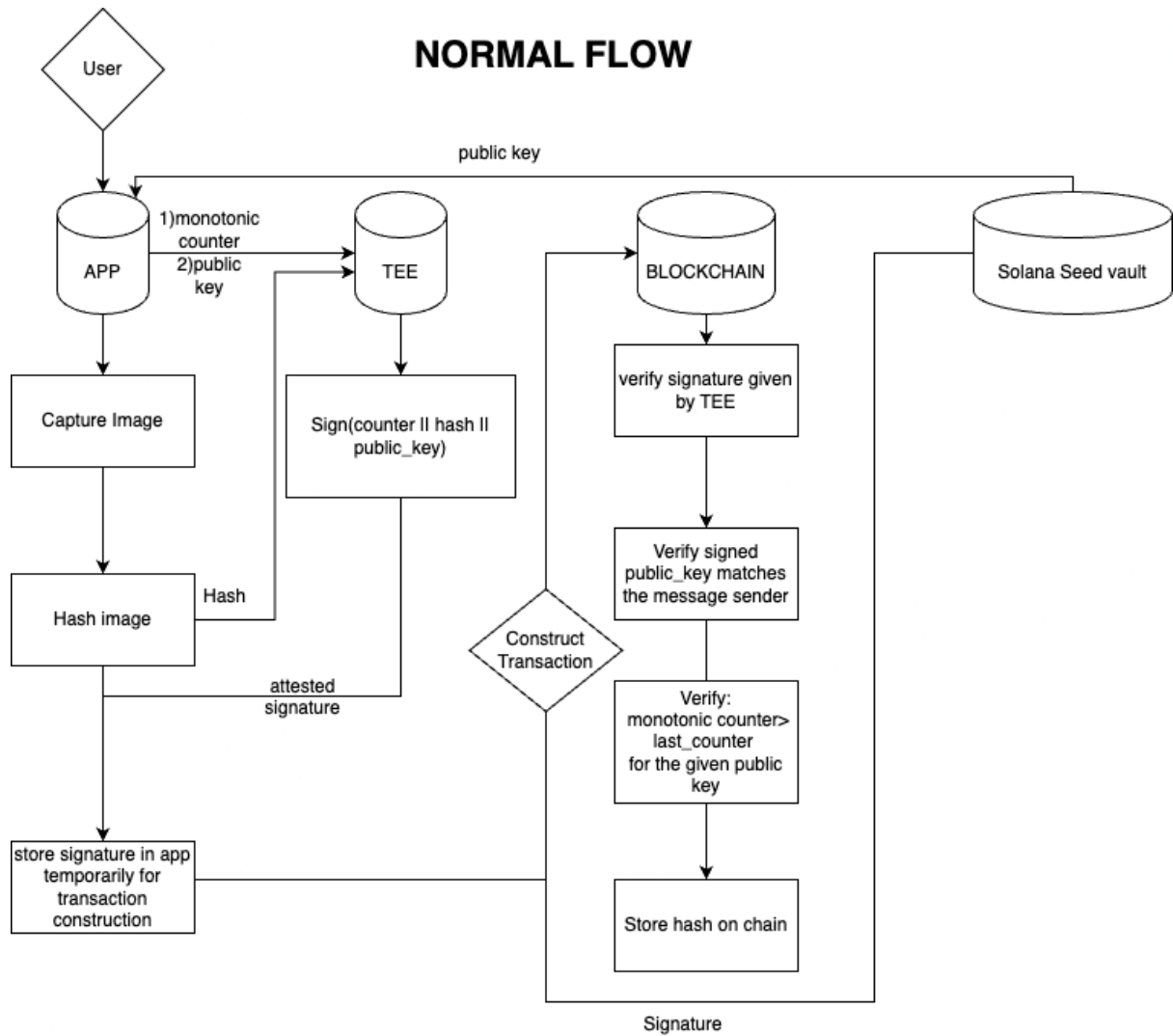
The Blockchain layer is where:

- A. Initial ZK-proof (used during app onboarding) proves the attested key belongs to the authorized app. Once verified, the key is stored and future transactions use direct signature verification.. The public key is stored in a mapping to the user's solana address.
- B. If the signature is valid for the stored public key, the image hash is recorded immutably on-chain as a verifiable event.
- C. On-chain replay protection is enforced using a simple per-pubkey counter. The smart contract verifies that the public_key exposed in the signature matches the message sender, and that the counter is strictly increasing for that identity. This ensures that proofs cannot be replayed or re-submitted from alternate wallets.

Here is a diagram showing a simple depiction of the process.

Key Initialisation & Attestation





ZK Circuit (Risc0) Overview

To enable on-chain verification, Risc0 STARK proofs are compressed into Groth16 format using the [Bonsol](#) zkVM bridge, allowing verification on Solana via built-in BN254 syscalls.

Based on the data we could find, verifying a Groth16 proof on Solana costs approximately 0.0025–0.0035 SOL (roughly 20–30 cents), depending on compute budget requirements and network conditions. *[cost calculations given after this section]*

Circuit Inputs:

- A. `attestation_cert_chain`: The full Android Keystore attestation certs in DER format(leaf,intermediate and root). This is a cert chain that is initialised on app installation.

Note: In this section, `public_key` refers to the solana wallet public key of the user.

Circuit Outputs:

- A. `bool is_valid_app`: True if attestation cert shows the correct package name & trusted root.
- B. `public_key`: The public key that the TEE attested.

Circuit Logic:

1. Parse and verify the certificate chain (up to Google Root CA).
 2. Check app identity: Validate the certificate includes your app's package name (can be extracted from the `teeEnforced` fields)
 3. extracts the attested public key, and exposes it as public an output.
 4. Output true/false flag
- Before a wallet address can be included in the signed payload, the app requires the user to cryptographically prove ownership of the address via a one-time signed message. This ensures that the TEE only signs proofs for addresses the user controls, eliminating identity spoofing and false attribution attacks.

Cost Note:

Verifier Key Storage Cost:

DecentraCam's on-chain Groth16 verifier includes a 20KB verifying key stored on-chain. This incurs a one-time cost of ~0.07–0.1 SOL to rent-exempt the storage account and keep it permanently available for verification. Each time the circuit is upgraded, a new verifier key will need to be deployed and re-funded. These costs are small but must be accounted for in governance and maintenance budgeting.

Gas Costs:

While Bonsol doesn't publish formal benchmarks, we estimate on-chain verification costs based on known Solana constraints and typical Groth16 proof characteristics. Groth16 proofs are ~128–256 bytes in size and fit well within Solana's 1232-byte transaction payload limit.

On Solana mainnet:

- **Base compute unit budget** per tx is 200,000
- **Groth16 verification** typically requires 150,000–180,000 CU
- **CU price per tx** varies, but ~0.00001 SOL/CU is common under load

Estimated verification cost:

150,000 CU × 0.00001 SOL/CU = 0.0015 SOL (low end)

180,000 CU × 0.000015 SOL/CU = 0.0027 SOL (under load)

Therefore, **the realistic mainnet cost is 0.0015–0.0027 SOL**, or roughly **\$0.22–\$0.40 USD**, assuming average network conditions. (assuming 1 Sol = 150\$)

These estimates are rough, and this requires testing on the network before we can confirm anything

Ram estimates

Risc0 >1 GB is for a small circuit; ours parses X.509 and does ECDSA verify → expected ~1.4 GB. Add "≥2 GB free RAM recommended".

Note: The current integration uses bonsol-solana, which is an open-source, proof-of-concept implementation. While it demonstrates end-to-end functionality, it may require security audits and performance hardening before production deployment.

Threat analysis

1. Image Tampering After Capture

- **Threat:** The user captures an image and later modifies it (e.g., Photoshop or AI-based edits), trying to pass it off as the original.
- **Mitigation:** The image hash is generated immediately after capture and signed by the TEE-backed attested key. Any modification, even a single pixel, changes the hash and invalidates the signature.

2. Faking the App Source

- **Threat:** An attacker builds a fake app that mimics DecentraCam and tries to submit forged proofs.
- **Mitigation:** The TEE-generated attestation proves the private key used to sign the hash is *bound* to the legitimate app's package name and signing cert. Only the real DecentraCam app can generate a valid attestation.

3. Replay Attacks

- **Threat:** An attacker reuses a past signature with a a)different image, b)or replays the same image repeatedly.
- **Mitigation:** a)Each signature is tied to a specific hash. Replaying an old signature with a new image will fail hash verification.
b)For every photo: sign (hash || monotonic_counter || public key). The blockchain verifies the sig + counter monotonicity, along with a public key match to the signer of the transaction.

4. Malicious Devices

- **Threat:** Attacker uses a rooted, compromised, or emulated device to forge attestation.
- **Mitigation:** Devices must provide attestation that's signed by a manufacturer-trusted root. The blockchain verifier (or ZK circuit) checks this chain of trust. Fake devices cannot produce valid attestation.

5. Privacy Leaks

- **Threat:** Sensitive metadata like GPS or device ID is leaked on-chain.
- **Mitigation:** The ZK-proof hides all private information and reveals yes/no: *Was the attestation valid? What is the ED25519 public key corresponding to the attestation?*

6. Compromised OS

- **Threat:** A Compromised or rooted OS poses a challenge for our application since a sufficiently experienced developer could feasibly alter the image captured by the camera before it reaches our application this allows the attacker to falsely authenticate an image
- **Mitigation:** The best current mitigation is to use the camera2 library, which is the latest and most secure method available to access the camera currently, avoiding 3rd party apps and directly hashing image after capture to reduce tamper. Set `android.hardware.security.level == STRONGBOX` and `timestampSource = REAL TIME`
Along with that add various metadata items such as the light intensity, focus etc. However these now lead to the same difficulties a normal image faces on authenticating, we can increase difficulty further by using verified boot and OS verification inside the Zkproof. However for far future versions we will use HAL camera attestation. This is discussed further in the roadmaps section.

Comparisons

Feature	Truepic Vision	DecentraCam
Image Verification	Centralized, black-box	Trustless, cryptographic
Storage	On company servers	Just the hash, stored immutably on-chain
Source Proof	Claimed via metadata	Enforced via TEE-bound attestation
Tamper Detection	Server-side forensics	Cryptographic proof from TEE
Privacy	Requires full image + metadata	Only the hash + ZK-proof are shared
Security Model	Trust the company	Trust the math (ZK + TEE)
Single Point of Failure	Yes	No

Feature	C2PA	Starling Lab	Worldcoin (PoP)	This System
Real-time capture binding	✗	⚠ Possible sometimes	✓	✓
Hardware-backed attestation	✓	✗	✓	✓
Zero-knowledge privacy	✗	✗	✗	✓
On-device proof generation	✗	⚠ Possible sometimes	✗	✓
Blockchain integration	⚠ Possible sometimes	✓	✓	✓
Consumer-grade hardware support	✗	✓	✗	✓
Decentralized trust	✗	✓	⚠ Possible sometimes	✓

C2PA (Coalition for Content Provenance and Authenticity) is an open standard led by Adobe and Microsoft for certifying digital media authenticity. It relies on a combination of device metadata, cryptographic signatures, and centralized certificate authorities. While C2PA defines a rich framework for image provenance, it is primarily designed for **enterprise workflows** and often depends on **cloud-based verification**, limiting its usability in decentralized or censorship-resistant environments. In contrast, our system is **fully on-device**, privacy-preserving via zero-knowledge proofs, and immutably records image hashes on a public blockchain with no reliance on third-party servers.

Starling Lab is a research initiative focused on using cryptography and decentralized infrastructure to preserve the integrity of digital media, particularly for human rights and journalism. Starling combines traditional cryptographic timestamping with archival tools and decentralized storage (e.g., IPFS, Filecoin). However, it typically assumes a **trusted capture context** and focuses more on **post-capture workflows**. Our system complements this by pushing trust all the way to the moment of capture, using **TEE-backed attestation and real-time image certification** — minimizing the trust surface at the origin.

Worldcoin Proof-of-Personhood, while unrelated to image authenticity directly, demonstrates the importance of **hardware-bound biometric attestation** in high-stakes decentralized systems. Like our approach, it leverages device-level attestation and cryptographic verification to prove that a certain event (in their case, a biometric scan) happened on a valid device. The key distinction is that our system proves the **authenticity of a captured moment**, not personhood, using open hardware (phones), not custom hardware (orb scanners).

Roadmap

Here we will discuss possible future innovations and routes our app can expand to, along with addressing various security concerns with the current design.

1.Future Innovations

For this application to be useful, we will also need various satellite applications such as a simple app which allows anyone to verify external images using simple api calls to the blockchain and hash comparisons. We can imagine a world where journalists who post various critical images are required to also include a lossless version of the image attached which anyone can download and verify for themselves its authenticity.

We can abstract out the concept this entire app is built around, see it as an app that authenticates data on capture. Any data.

Once we realise this, the applications of this same core pipeline are wide. Anything from audio/video capture to data capture in IOT devices can benefit with this verification. Between the two, audio and video capture are currently more feasible and most likely going to be the next feature since IOT devices have a long way to grow before they can be integrated.

Prover Strategy & UX Tradeoff:

In Phase 1, DecentraCam uses Risc0 to generate zero-knowledge proofs entirely on-device. While this ensures full trust and privacy, it imposes computational overhead — proof generation may take 30–60 seconds per image and use over 1GB of RAM. As a result, the user experience requires waiting after each capture before proceeding.

2.Threats

Currently, the most dangerous vulnerability comes from a rooted OS, wherein the untrusted OS could feed the app incorrect images which pass authentication. The long term solution is Integration of **Camera HAL attestation** which will cryptographically prove that the captured image originated directly from the sensor and was signed by a secure hardware-backed key, bypassing the user space entirely. This renders the application almost completely secure. While the system defends against all known software-level attacks, we acknowledge that absolute hardware security is probabilistic. Our goal is to raise the cost of forgery beyond the capabilities of any realistic attacker.

However, this technology has yet to be incorporated into phones, and currently similar concepts are only available in a few camera's such as Leica M11-P.

Until then, we can achieve reasonable security against such attacks by using a verified boot and verifying the OS version within the ZK-proof.

Device Identity Reuse (Non-Critical Sybil Vector):

A user can currently uninstall and reinstall the app to reset their key and counter, allowing them to appear as a new device using a different wallet. While this doesn't affect authenticity guarantees, it can weaken proof uniqueness and enable Sybil-like farming.

Future versions will mitigate this via persistent key reuse, using Android Keystore's ability to detect existing attested keys created by the same app package, and by tracking counters per device key instead of per wallet.

We may also introduce a small on-chain posting bond to discourage Sybil spam. Bonds are refunded upon valid submission and slashed for invalid or malicious proofs.

3.General Roadmap:

Phase 1

1. Image hashing + TEE signature
2. ZK circuit validating attestation cert
3. Solana integration with Groth16 via Bonsol

Phase 2

1. Implement TEE key persistence, checking for existing keys on app launch to prevent reset-based Sybil replay.
2. Shift counter tracking to a device-fingerprint model, binding uniqueness to hardware instead of wallet address.
3. CLI + mobile verification tools
4. Basic explorer frontend for journalists and courts

Phase 3

1. Verified boot + OS version parsing inside ZK circuit, this will alter pipeline slightly.
2. Add **privacy-preserving pseudonym system**, replacing direct pubkey and counter outputs with hashed aliases to prevent chain-wide activity tracking.

Conclusion

We have proposed a system of capturing and authentication images without relying on trust in any one centralised system. We start with the usual framework of storing hashes to prove the authenticity of certain data without storing the data, which is incomplete without a reliable way to verify if the source of the hash is the trusted, open-sourced application.

This gap is addressed through the use of Trusted Execution Environments (TEEs), which provide a cryptographic attestation of origin, bound to the app's identity and device integrity. To reduce the computational burden on blockchain validators and preserve user privacy, we further employ zero-knowledge proofs that validate the attestation without revealing sensitive device-level details.

The result is a decentralized, tamper-proof witness mechanism that transforms a simple smartphone camera into a cryptographic source of truth.... a **cryptographic witness**.

CITATIONS

1. Truepic Vision's Centralized Storage:

- <https://www.truepic.com/blog/remote-inspections>

2. C2PA's Centralized Approach to Content Authenticity:

- <https://c2pa.org/>

3. Starling Lab's Post-Capture Verification Focus:

- The Starling Lab. (n.d.). *The Starling Lab for Data Integrity*
<https://www.starlinglab.org/>

4. Worldcoin's Biometric Proof-of-Personhood System:

- Proof of personhood
<https://whitepaper.world.org/#building-blocks>

5. Bonsol: Solana-Native Verifiable Computation Framework

- Bonsol. (n.d.). *Bonsol: ZK Co-Processor for Solana*.
<https://bonsol.sh/docs/explanation/what-is-bonsol>
- Bonsol Collective. (n.d.). *bonsol: Solana + RISC0*. GitHub
<https://github.com/Bonsol-Collective/bonsol>
- <https://solanacompass.com/learn/breakpoint-24/breakpoint-2024-product-key-note-bonsol-zk-plumbing-on-solana-austin-adams>